SQL QUERY GENERATOR

Natural Language to SQL with IBM Granite 3.3 2B

Team Leader:M.Sakthi Sobika

Team Member:S.Samaya Sabini

Team Member:S.Sameeha Nachiyar

Team Member:S.Sangavi

Team Member:K.Santhini

Project Description

SQL Query Generator is an intelligent application that converts natural language requests into SQL queries.
The goal of the system is to help users interact with databases without requiring deep knowledge of SQL syntax.

The system uses a machine learning language model to understand the user's request written in simple English.
Based on predefined database schemas, the application generates appropriate SQL statements that can be used to retrieve data.

The platform is built using Python and provides an interactive interface using Gradio.
Users can enter natural language queries,
generate SQL queries, view query history, and export results as text or PDF files.

The system improves accessibility to databases and demonstrates how artificial intelligence can simplify data retrieval tasks.

Scenario 1: User Analytics Query

In this scenario, a user wants to identify all users who recently registered in the system.
The user types a request such as 'Show all users who signed up last month'.

The system analyzes the request using the AI model and detects the relevant table, which is the users table.
It generates an SQL query that retrieves user information filtered by signup date.

The generated query is displayed in the output console and saved in the query history for future reference.

Scenario 2: Transaction Filtering

In this use case, a user requests financial transaction records based on a specific condition.
For example, the user may ask: 'Find transactions greater than 500'.

The system identifies the transactions table and detects the numeric condition for filtering.
An SQL query is generated with a WHERE clause that retrieves transactions whose amount is greater than the specified value.

Scenario 3: Export and Reporting

The system allows users to export generated SQL queries for documentation and sharing purposes.
Users can download the query history as a TXT file or as a formatted PDF report.

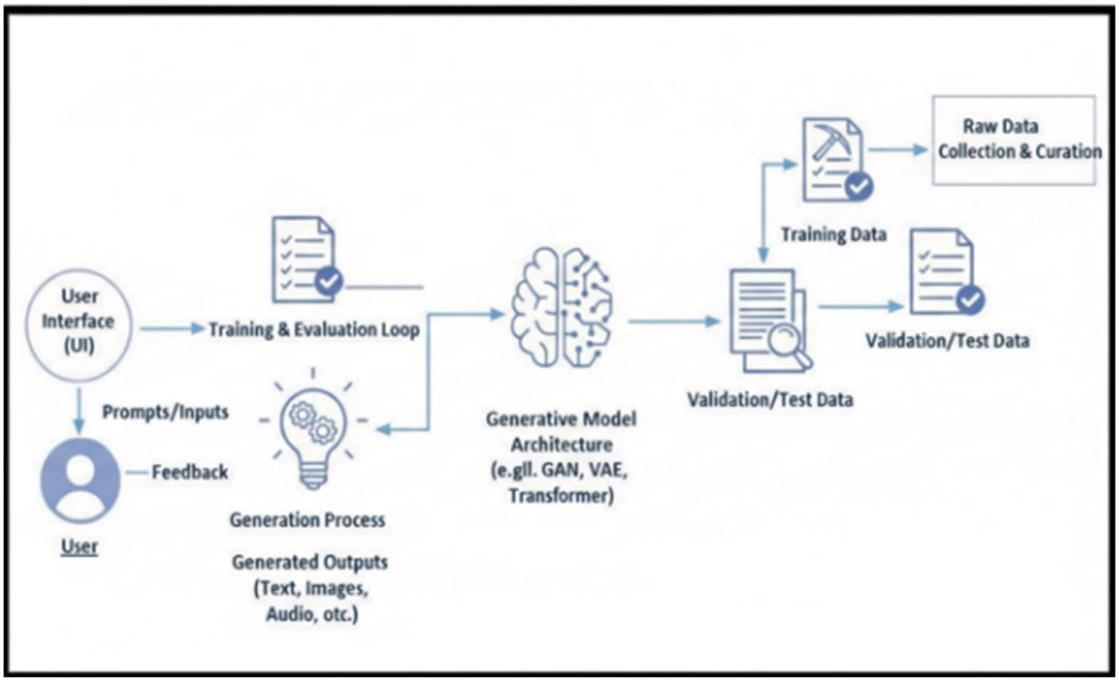
This feature is helpful for project documentation, collaboration with team members, and maintaining a record of generated queries.

Technical Architecture

The technical architecture integrates the AI model with the user interface and SQL generation engine.

User Input → Natural Language Processing → SQL Generation Engine → Output Display → Export System

This architecture ensures that natural language input is processed efficiently and converted into accurate SQL queries.



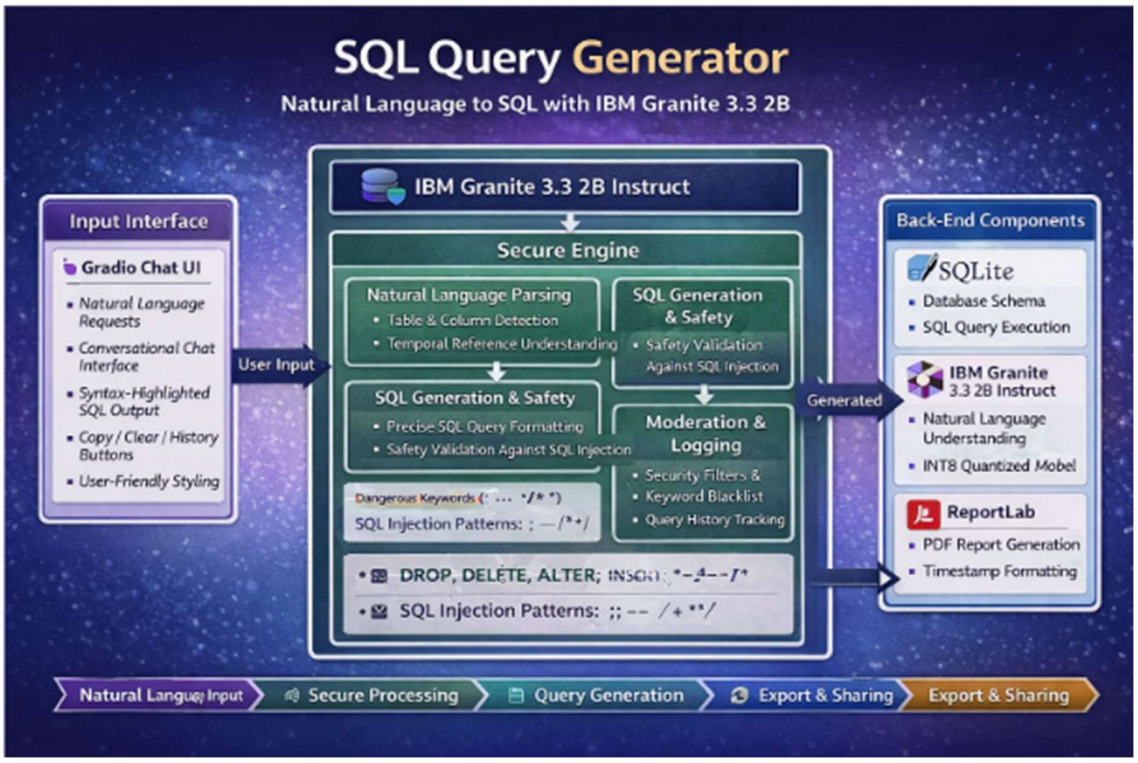
Architecture Overview

The SQL Query Generator follows a modular architecture combining AI processing with an interactive user interface.

The system includes the following main components:
• User Interface – Accepts natural language input from users.
• Natural Language Processing Engine – Interprets the user's request.
• SQL Query Generation Engine – Generates SQL statements.
• Database Schema Module – Defines tables and columns used in query generation.
• Query History Module – Stores previously generated queries.
• Export Module – Allows downloading queries in TXT or PDF format.

These components work together to create a simple and efficient natural language database query system.

Project Structure:



Project Workflow:

Phase 1: Environment Setup

Activity 1.1: Install Dependencies

!pip install gradio transformers torch fpdf

This installs the required libraries for the project.

Phase 2: AI Model Integration

The system loads the CodeGen AI model using Hugging Face Transformers.

generator = pipeline("text-generation", model="Salesforce/codegen-350M-mono")

This model generates SQL queries from natural language prompts.

Phase 3: SQL Query Generation Engine

The function generate_sql() converts natural language into SQL queries.

Steps:

1. **User enters request**
2. **Prompt is created using database schema**
3. **AI model generates SQL query**
4. **Query is stored in history**
5. **Result is displayed in console**

Phase 4: Query History Management

The system stores all generated SQL queries in a history list.

Functions include:

- **clear_history() – Clears all stored queries**
- **display history – Shows previous queries in console**

Database Schema

Table: users

id: INTEGER PRIMARY KEY

name: TEXT

email: TEXT

signup_date: DATE

age: INTEGER

country: TEXT

status: TEXT

Table: orders

id: INTEGER PRIMARY KEY

user_id: INTEGER (Foreign Key to users.id)

product_name: TEXT

amount: REAL

order_date: DATE

status: TEXT

Table: products

id: INTEGER PRIMARY KEY

name: TEXT

price: REAL

category: TEXT

stock: INTEGER

Table: transactions

id: INTEGER PRIMARY KEY

user_id: INTEGER (Foreign Key to users.id)

amount: REAL

date: DATE

type: TEXT

Core Technologies

Python – Core programming language for application development
Gradio – Web interface framework for building the interactive UI
Hugging Face Transformers – Library used for loading AI models
CodeGen Language Model – Used for natural language to SQL generation
PyTorch – Deep learning framework used by the model
FPDF – Library used to generate PDF reports

Pre-requisites

Software Requirements:
Python 3.8 or higher
Gradio library
Transformers library
Torch (PyTorch)
FPDF library
Internet connection to download AI models

Knowledge Prerequisites

Basic Python programming knowledge
Basic SQL query understanding
Knowledge of databases and tables
Understanding of machine learning models

Hardware Requirements

Processor: Intel i3 or higher
RAM: Minimum 4GB
Storage: Minimum 1GB free space
Web browser: Chrome, Edge, Firefox or Safari

Project Workflow

Phase 1:

Environment Setup and Model Configuration
Install required libraries and configure the AI model.

Phase 2:

Core Engine Development
Implement the SQL generation logic and database schema definitions.

Phase 3:

Interface Development
Create the Gradio-based user interface for accepting user input and displaying generated SQL queries.

Phase 4:

Integration and Testing

Integrate all components and test the system using different user queries.

Phase 5:

Deployment and Documentation

Deploy the application and prepare the final project documentation.

Milestone 1: Environment Setup

Install required dependencies:

```
!pip install gradio transformers torch fpdf
```

Milestone 2: AI Model Integration

Load the AI model using Hugging Face transformers.

The model analyzes user input and generates SQL queries based on the database schema.

```
def generate_sql(question):

    prompt = f"""
Convert the natural language request into SQL query.

Database schema:
{schema}

Request: {question}

SQL Query:
"""

    result = generator(prompt, max_length=120)

    text = result[0]["generated_text"]
    sql = text.split("SQL Query:")[-1].strip()

    history.append(sql)

    return "\n".join(history)
```

Milestone 3: Query Generation Engine

Implement functions that convert natural language into SQL queries.

The system processes the user input, detects

Example stored queries:

SELECT * FROM users;
SELECT * FROM transactions;
SELECT SUM(amount) FROM transactions;

This allows users to **track previously generated SQL queries**.

keywords, and generates SQL commands.

Export TXT

Python

```python
def download_txt():
    with open("query.txt","w") as f:
        for q in history:
            f.write(q+"\n")
```

Export PDF

Python

```python
def download_pdf():
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial", size=12)

    for q in history:
        pdf.cell(200,10,txt=q,ln=True)

    pdf.output("query.pdf")
```

Milestone 4: Interface Development

Objective

The objective of this milestone is to develop a **user-friendly interface** that allows users to interact with the SQL Query Generator system. The interface enables users to input natural language queries, generate SQL statements, view query history, and export the generated queries into files.

The interface for this project is developed using **Gradio**, which allows quick creation of interactive web applications for machine learning models.

Create the Gradio interface with:

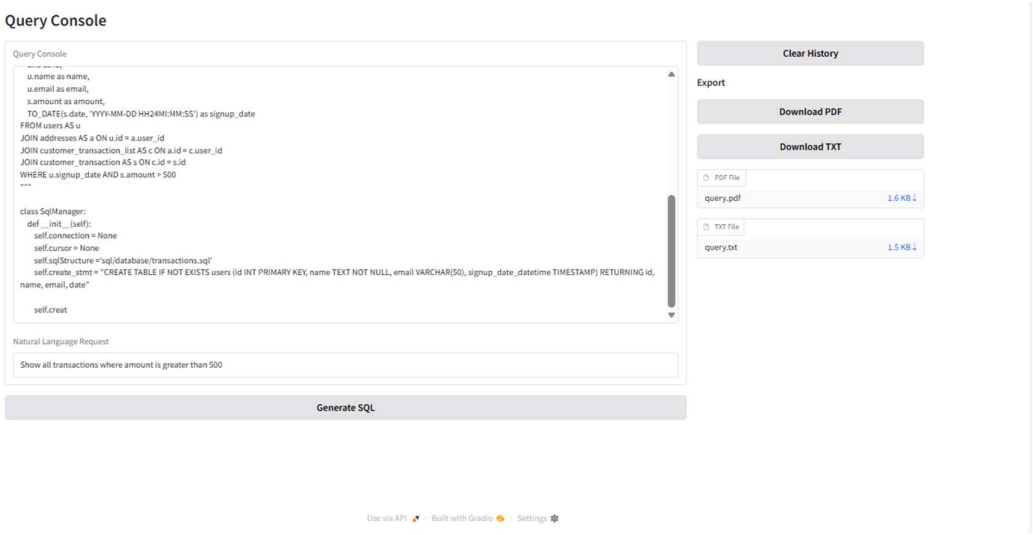
- Input textbox
- Generate SQL button
- Query console
- Download options
- Clear history button

```
with gr.Blocks() as demo:

    gr.Markdown("# Query Console")

    with gr.Row():

        with gr.Column(scale=3):
            console = gr.Textbox(lines=15,label="Query Console")

            inp = gr.Textbox(label="Natural Language Request")

            btn = gr.Button("Generate SQL")

        with gr.Column(scale=1):

            clear_btn = gr.Button("Clear History")

            gr.Markdown("### Export")

            pdf_btn = gr.Button("Download PDF")
            txt_btn = gr.Button("Download TXT")

            pdf_file = gr.File(label="PDF File")
            txt_file = gr.File(label="TXT File")
```

Milestone 5: Testing and Deployment

Test the system with multiple queries and verify that correct SQL queries are generated.

Deploy the application using Gradio so users can access it through a web interface.



Functional Testing

Test Case 1

Input: Show all users
Output: SELECT * FROM users;

Test Case 2

Input: Show transactions greater than 500
Output: SELECT * FROM transactions WHERE amount > 500;

Conclusion

SQL Query Generator successfully delivers an intelligent database querying assistant that bridges the gap between natural language and SQL. By leveraging IBM Granite 3.3 2B Instruct model's advanced language understanding and combining it with comprehensive security validation, the system provides safe, accurate, and production-ready SQL queries from plain English requests.

The platform demonstrates the power of AI-assisted database operations, making SQL accessible to non-technical users while maintaining enterprise-grade security through multi-layered protection against SQL injection and malicious queries. With professional export capabilities including PDF reports, text files, and social media sharing, the system serves both individual analysts and collaborative teams.

Through its intuitive Gradio chat interface, intelligent natural language processing, and comprehensive feature set, SQL Query Generator transforms database querying from a technical barrier into an accessible, conversational experience that empowers users to extract insights from data without requiring SQL expertise.

Future enhancements could include support for JOIN operations across multiple tables, GROUP BY and aggregation functions, custom database schema uploading, query optimization suggestions, and integration with popular database management systems beyond SQLite. The modular architecture allows for seamless extension and customization to meet evolving user needs and database requirements.